Verify Before You Execute: When Sound Checking Beats LLM Judging for Pre-Execution Plan Verification

Ari Bajaj
Illinois Mathematics and Science Academy
abajaj1@imsa.edu

Abstract

LLM agents increasingly emit multi-step plans—action sequences, tool-call chains—that downstream systems execute directly, where a flawed plan is paid for at execution time. We present a pre-execution verifier that translates a free-text plan into a typed action schema and *soundly* checks consistency, goal completeness, and resource feasibility by forward simulation. We initially hypothesized that its advantage over an LLM "plan judge" was accuracy that widens with plan length—and with a small judge model, judge recall did fall from 0.94 to 0.84 as plans grew from 10 to 80 steps while the checker held recall 1.0. **A stronger frontier judge overturned that explanation:** under the identical prompt and token budget it showed no measurable horizon-dependent degradation, scoring recall 1.0 at 20, 40, and 80 steps alike. Horizon-driven judge degradation is thus a property of weaker judges, not of judging. What survives is a sharper claim, stated as a family of propositions and proved from an expected-cost model: whenever a judge merely *matches* the checker's accuracy, the checker is still expected-cost-optimal because it is sound at *zero* marginal decision-time cost—dominance by cost and guarantee, not by accuracy. Empirically the checker is expected-cost-optimal across the entire (stakes, cost-weight) grid at every horizon ≥ 10; at 3–8 steps, where extraction noise gives the checker a small false-reject rate, a cheap accurate judge can instead be optimal—so the boundary is horizon- and cost-dependent, which we map. The deployment argument needs no model comparison at all: the checker is structurally immune to the silent guardrail-collapse modes we document, in which a fail-open LLM judge with a too-small token budget accepted 100% of flawed plans while appearing to run. We validate the pipeline on three synthetic domains and on real tasks from the ALFWorld benchmark (65% of sampled tasks translate onto the schema, with the independent-oracle cross-check holding on every translated task). All results reproduce from a single make target on a laptop.

1 Introduction

A growing class of systems lets a large language model (LLM) propose a multi-step plan—a sequence of robot actions, API calls, or tool invocations—and then executes that plan with little or no intermediate checking. When the plan is flawed, the cost is paid at execution time: side-effecting API calls fire, budgets are spent, and physical or irreversible actions occur before the flaw surfaces. Benchmarks consistently find that LLM plans are unreliable in exactly the ways that matter: preconditions are violated, goal conjuncts are silently dropped, and resource limits are exceeded (Valmeekam et al. 2023; Kambhampati et al. 2024).

The natural safeguards occupy two unsatisfying extremes. Purely symbolic validation in the style of VAL (Howey, Long, and Fox 2004) is sound but assumes the plan already exists in a formal language—while LLM plans arrive as free text, and the lossy translation from text to formalism becomes an unmodeled failure source. Purely learned safeguards—asking an LLM judge "is this plan valid?" (Zheng et al. 2023)—inherit the fallibility they are meant to check, and recent evidence suggests LLMs are poor self-verifiers (Huang et al. 2024; Stechly, Valmeekam, and Kambhampati 2025).

We study the middle: an LLM extractor translates each free-text step into a typed action schema, and a sound symbolic checker forward-simulates the extracted plan and verdicts three independent dimensions (action consistency, goal completeness, resource feasibility). The question this paper actually answers is not "does this beat an LLM judge on accuracy" but *when* a sound-but-narrow checker is the better verification strategy than an approximate-but-general judge—a decision-theoretic question in the stakes of a missed flaw, the cost of an unnecessary replan, and the dollar cost of a judge call.

An honest reversal. Our first hypothesis was that the checker's edge is accuracy that grows with plan length. With Claude Haiku as the judge, the data seemed to confirm it (judge recall $0.94 \rightarrow 0.84$ over $10 \rightarrow 80$ steps; checker 1.0 throughout). Re-running the identical experiment with a stronger judge, Claude Sonnet 5, refuted the explanation: Sonnet held recall 1.0 at 20, 40, and 80 steps. The degradation was a property of the weaker *judge*, not of judging with horizon. We report this in full and rebuild the contribution on what survives it.

Contributions. (1) A sound pre-execution verifier for *free-text* LLM plans, with a synthetic-data methodology anchored by an *independent* oracle: two separately implemented simulators agree on every record (360/360; 1,440/1,440 per-dimension comparisons). (2) The reversal above, reported as a first-class result: horizon-driven judge degradation did

not replicate for a stronger judge, so we do *not* claim it as a general property of judging. (3) A decision-theoretic account with a proved proposition: when a judge only matches the checker's accuracy, the checker is still expected-cost-optimal because it is sound at *zero* marginal cost; empirically this holds across the whole (stakes, cost-weight) grid at every horizon ≥ 10, while at 3–8 steps a cheap accurate judge can win because extraction noise gives the checker a small false-reject rate—a horizon- and cost-dependent boundary we map rather than assert. (4) A generality test: a tool-use domain (auth scopes, prerequisite calls, quotas, spend budgets) ran through the entire pipeline with *zero* code changes. (5) A tier-independent deployment argument: a taxonomy of silent guardrail-collapse modes—including a fail-open judge that accepted 100% of flawed plans while appearing to run—to which the checker is structurally immune. (6) A negative result reported as such: a learned trust layer over the checker's parse was inert under *two* independent calibration methods at our data scale, for a diagnosable reason.

2 Related Work

LLM planning and its critics. PlanBench and successors document systematic planning failures in LLMs (Valmeekam et al. 2023). The LLM-Modulo position argues LLMs should generate while external sound modules verify (Kambhampati et al. 2024); LLM+P (Liu et al. 2023) translates problems into PDDL and delegates to a classical planner. We keep the LLM as planner and verify its output, targeting settings (e.g. tool use) where a complete classical model exists only implicitly.

Plan validation. VAL (Howey, Long, and Fox 2004) validates PDDL2.1 (Fox and Long 2003) plans against a formal domain. Our symbolic core is a deliberately small reimplementation of this idea over a typed DSL with per-step resource accounting; the novelty is not the checker but its coupling to a learned model of *whether the checker saw the right plan*.

LLM self-checking and repair. LLM-as-judge (Zheng et al. 2023), Self-Refine (Madaan et al. 2023), and Reflexion (Shinn et al. 2023) use LLMs to critique or repair their own outputs; Huang et al. (2024) and Stechly, Valmeekam, and Kambhampati (2025) find intrinsic self-correction weak on reasoning and planning. Our baselines instantiate both judge variants and a Reflexion-style repairer on identical inputs, and our downstream experiment measures what each safeguard is worth at execution time.

Calibration and self-consistency. We use k-resample agreement in the spirit of self-consistency (Wang et al. 2023) as a *feature* rather than a decoding rule, Platt scaling (Platt 1999) for calibration, and expected calibration error (Naeini, Cooper, and Hauskrecht 2015; Guo et al. 2017) for evaluation.

3 Method

3.1 Plan and Domain Representation

A domain is a set of typed predicate definitions, action schemas (preconditions, add/delete effects), and *resource dimensions*—named scalars with an initial value, a floor, and a cap, adjusted by per-action deltas. A problem grounds a domain with typed objects, an initial state, and a conjunctive goal. Validation is structural: an action schema may only reference declared predicates and resources with matching arity and types. This DSL is intentionally small—no disjunctive preconditions, conditional effects, or typed return values—but Section 5.9 shows it stretches to tool-use constraints without modification.

3.2 Symbolic Verifier

Given a sequence of ground actions, the verifier forward-simulates from the initial state and emits a three-part verdict with a machine-readable explanation: **consistency** (every step's action exists, is well-typed, and its preconditions hold when executed in order), **goal completeness** (every goal conjunct holds in the final state), and **resource feasibility** (every prefix of the running sum respects every dimension's floor and cap). Progression is *lenient*: a violated precondition is recorded but effects still apply, so one early slip cannot mask an independent later flaw—each dimension verdicts independently. Upstream parse or extraction failures are injected as consistency violations, since an executor could not run those steps. `overall_valid` is the conjunction of the three checks.

3.3 NL→Schema Extraction with Self-Consistency

Free-text steps are mapped to ground actions by an LLM extractor using the API's structured-output mode, validated against the domain (action name, arity, object types). On validation failure the extractor retries once with the error as feedback, then falls back to a null extraction with confidence 0—it never crashes, it degrades. Each step is resampled $k{=}3$ times at temperature 0.7; the modal extraction is used, and two agreement statistics (exact-match fraction and action-type-match fraction) are retained. These, with the extractor's self-reported confidence, become features rather than being discarded.

3.4 Fusion Rule and an Optional Trust Layer

Fusion is a two-clause rule with a tested invariant:

$$\textbf{reject} \iff \neg\text{symbolic\_valid} \ \vee \ \text{trust} < \tau,$$

where the first clause is unconditional—no trust score can rescue a hard symbolic failure—and τ is picked on validation ($\tau{=}0$ recovers symbolic-only exactly). The optional *trust* score is a calibrated logistic model predicting whether the extraction's verdict is faithful to the plan as written (label: agreement with a rule-based reference parse), over extraction-confidence, k-resample-agreement, plan-shape, and symbolic-verdict features; the full 17-feature list and training details are in the repository. We include it for completeness, but—previewing Section 5.4—it is reported as a *negative result* (inert at our data scale under two calibration methods), so we keep its footprint here minimal.

4 Experimental Setup

4.1 Domains, Problems, and Gold Plans

Blocksworld (4 operators + a crane-energy resource), **Logistics** (trucks/planes over two cities, fuel + budget), and **Tools** (Section 5.9). A seeded generator produces problems whose BFS-optimal gold plans span 3–8 steps; determinism is enforced cross-process (a `PYTHONHASHSEED` ordering bug found by a subprocess test motivated sorting all set-derived output). Resource caps are tightened to $\lceil 1.25\times \rceil$ the gold plan's consumption so that wasteful plans can actually overrun.

4.2 Long-Horizon Problems

Prior LLM-planning benchmarks concentrate on short plans, and our own main dataset (3–8 steps) is no exception—raising the question of whether any detector comparison run at that length generalizes. To test this directly we build a second, constructive generator per domain (BFS gold-planning is infeasible past $\sim$10 steps) that scales problem size (blocks, packages, trips) to hit nominal horizons of 10, 20, 40, and 80 reference-plan steps: Blocksworld tears every tower down to the table and rebuilds the goal arrangement bottom-up; Logistics routes each package sequentially through its truck/airport/plane legs; Tools authenticates each needed scope once and executes every trip's milestone prerequisite chain in order. Every constructive plan is independently strict-simulated at generation time and re-checked by the oracle labeler, so a bug in a constructive planner cannot silently ship an invalid reference plan; the one honest caveat is that these plans are valid but *not optimal* (Blocksworld especially), so "horizon H" denotes a band on the reference length, not an optimality claim. To bound API cost, the k-resampled extraction path runs on all 40 records per horizon at $H\in\{10, 20\}$ but on a stratified 12-record subset at $H\in\{40, 80\}$; judges and self-repair run on all records at every horizon. Token budgets were raised throughout (generation/paraphrase to 4,096, judges/self-repair to 8,192 tokens) so long plans are not silently truncated the way Section 5.6 describes.

4.3 Plan Generation with Failure Injection

For each problem, `claude-haiku-4-5` (temperature 1.0) writes a plan under four prompt conditions: *baseline* (honest prompt), *goal-omission* (one goal conjunct silently dropped from the prompt; labels use the full goal), *resource-blind* (limits stripped), and *distractor* (plausible invalid actions offered). A pilot calibration confirmed the conditions amplify naturally occurring failure modes rather than inject alien ones: 43–47% of *baseline* plans are already flawed, and goal-omission roughly quadruples goal-incompleteness. Dataset: 3 domains $\times$ 30 problems $\times$ 4 conditions $=$ 360 records.

4.4 Independent Oracle and Soundness Evidence

Ground-truth labels come from an oracle labeler that is a *separate implementation* of plan simulation from the verifier (shared design semantics, disjoint code). Agreement between the two on rule-parsed plans is therefore evidence, not tautology: across all 360 records they agree on overall validity 360/360 and on all 1,440 per-dimension comparisons, with every flaw type additionally covered by hand-verified adversarial fixtures. The evaluation CLI recomputes this cross-check on every run and fails loudly on any disagreement.

System	P	R	F1
Hybrid (ours, τ picked on val)	0.903	**1.000**	0.949
Symbolic-only	0.903	**1.000**	0.949
Learned-only (no symbolic feats)	0.618	0.750	0.677
LLM judge, zero-shot	**1.000**	**1.000**	**1.000**
LLM judge, CoT	**1.000**	**1.000**	**1.000**
Self-repair (change detection)	0.966	**1.000**	0.982

Table 1: Detection on the 72-record pooled test split (positive class = flawed). Per-flaw recall for the hybrid: 20/20 inconsistency, 10/10 goal-incompleteness, 5/5 resource-infeasibility.

4.5 The Prose Regime

Our first finding is methodological: with a constrained output format (`Step` N`: action(args)`), the LLM extraction path's verdict agreed with the rule-based reference on **240/240** records—the NL$\rightarrow$schema translation was lossless, the trust label had one class, and the learned layer had nothing to model. This is a genuine scoping result: *where plans can be forced into a rigid format, a rule parser plus a sound checker suffices*. To study the setting the trust model exists for, the production pipeline verifies an LLM-generated *prose paraphrase* of each plan (one sentence per step, no rigid syntax), while labels stay anchored to the original constrained text. In the prose regime 13/360 records (3.6%) get unfaithful verdicts—sparse, but a real target class.

4.6 Baselines

On identical prose inputs: **LLM judge** (zero-shot and chain-of-thought, verdict parsed from a required `VERDICT:` line, fail-open on unparseable output, mirroring naive deployment); **symbolic-only** (τ=0); **learned-only** (the trust feature vector with all five symbolic-verdict features removed, trained to predict validity directly—the no-grounding ablation); and **self-repair** (Reflexion-style single-pass critique+rewrite (Shinn et al. 2023), with change-detection standing in for a verdict).

4.7 Metrics

Positive class = *flawed plan*; rejecting is a positive prediction. We report P/R/F1 overall, per domain, and per flaw type; ECE with reliability diagrams for the trust model; API-call and local-latency cost proxies; and the downstream experiment of Section 5.5. Test split: 72 records (problem-level, 216/72/72).

5 Results

5.1 Detection

Table 1 contains a result we report at full prominence: **both LLM judges are perfect on this test split**. Plans of 3–8 steps are short enough for the judge model to simulate flawlessly when given an adequate token budget (see Section 5.6). At this horizon "the checker beats an LLM judge on detection"

Horizon	Checker R	Haiku-zs R	Haiku-CoT R	Sonnet-zs R
5 (ref.)	1.00	1.00	1.00	—
10	**1.00**	0.94	0.92	—
20	**1.00**	0.91	0.95	**1.00**
40	**1.00**	0.88	0.93	**1.00**
80	**1.00**	0.84	0.88	**1.00**

Table 2: Flaw-detection recall vs. nominal plan horizon, pooled over domains. The weak judge (Haiku) degrades monotonically; the *stronger* judge (Sonnet-5), under the identical prompt and 8192-token budget, matches the checker at recall 1.0 at every horizon tested. Sonnet-5 was run at 20/40/80 (a targeted transfer check, zero-shot; the 40/80 cells use the 12-record-per-domain subset).

is simply *false*: a well-budgeted judge ties it. This is where we first expected longer plans to separate the two on accuracy—Section 5.2 tests that and finds it holds only for a weaker judge, not a stronger one, which is what redirects the paper's claim away from accuracy and toward cost and soundness. The learned-only ablation (F1 0.677) confirms the symbolic grounding, not the learned layer, carries the detection signal.

The checker's only errors here are 3 false rejects, all on the Tools domain (precision there 0.833, perfect elsewhere), all with one signature: the extractor mangles hyphenated arguments (`flights-scope` → `flights`), producing unknown-object violations on genuinely valid plans—the same extraction noise that gives the checker its nonzero false-reject rate at $H{=}5$ in the decision model (Section 5.3).

5.2 Judge Degradation with Horizon Is Model-Specific, Not a Property of Judging

We repeat detection at nominal horizons of 10–80 steps (Section 4.2). Our initial reading of Haiku alone (Table 2, columns 2–3; Figure 1) was that judging degrades with length: Haiku zero-shot recall falls $1.00 \to 0.94 \to 0.91 \to 0.88 \to 0.84$ and CoT drifts down similarly, and this is a genuine miss-the-flaw failure, not a parsing one (0/480 Haiku calls were unparseable and mean output tokens actually *rise* with horizon, $634 \to 854$ zero-shot). The checker, by contrast, holds recall 1.0 everywhere—expected, since forward simulation is linear in length and does not lose accuracy.

A stronger judge overturns the "degrades with horizon" reading. Running the identical experiment with Claude Sonnet 5—same prompts, same budget—gives recall **1.0 at 20, 40, and 80 steps** (Table 2, last column), with zero parse failures. The degradation is therefore a property of the weaker judge *model*, not of judging long plans per se, and we do not claim the general version. This matters because it changes what the checker's advantage *is*: not higher accuracy than any judge (a strong judge ties it), but the properties developed next—sound coverage at zero marginal cost (Section 5.3) and immunity to guardrail collapse (Section 5.6). The flaw rate among generated plans does rise with horizon (38.9% at $H{=}5$ to 100% at $H{=}80$), so longer plans are both more failure-prone and, for a weak judge, harder to check—but that last clause is judge-specific.

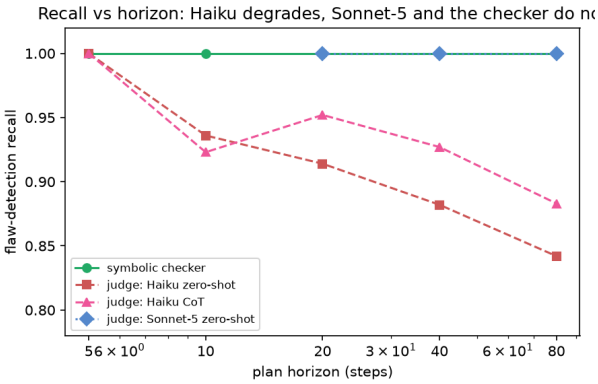

Figure 1: Flaw-detection recall vs. plan horizon, pooled. The checker stays at 1.0; the weak judge (Haiku) drifts downward, but the stronger judge (Sonnet-5) also stays at 1.0 at every horizon it was tested—so the downward drift is Haiku-specific.

The soundness gap this implies is not merely theoretical: we re-ran the downstream execute-or-reject experiment (Section 5.5) at $H \in \{10, 20\}$ (120 records each; $H{=}40, 80$ were not affordable to run downstream given the recall trend already visible at $H \leq 20$). At $H{=}5$ both gates achieved 0% executed-flawed. At $H{=}10$ the judge-gated arm (Haiku CoT, the pipeline's judge) lets **5%** of records execute a flawed plan despite the gate; at $H{=}20$, **4.2%**. The checker-gated arm remains at **exactly 0%** executed-flawed at both horizons. This is the crucial asymmetry: a sound gate *cannot* leak a flawed plan by construction, whereas any approximate gate can, and here does. A stronger judge would leak less (Section 5.2), but "less" is not "never"—only soundness gives never, at zero marginal cost. Task success falls for every arm as horizon grows (the underlying planner LLM struggles at 10–20 steps, independent of verification), but the hybrid still leads at both horizons (0.667 vs. 0.525 at $H{=}10$; 0.383 vs. 0.283 at $H{=}20$). Two limits on this result: the long-horizon extraction cells are 12-record stratified subsets with correspondingly wide error bars, and the constructive gold plans are valid but not optimal (Section 4.2).

5.3 Decision-Theoretic Account: When the Checker Wins

If a strong judge matches the checker's accuracy, what makes the checker the better *choice*? We answer with an expected-cost model over three measured quantities per strategy at a horizon: its miss rate $1 - \text{recall}$ (each miss costs the stakes S of executing a flawed plan), its false-reject rate (each costs a replan r), and its real per-record dollar call cost (weighted by λ, converting dollars into the same units):

$$\text{cost} = (1 - \text{recall})\, S + \text{fr} \cdot r + \text{call\_cost} \cdot \lambda.$$

The checker's call cost is 0 (no API call at decision time); a judge's is its measured token cost times list price.

We give a family of three propositions, one per regime the data exhibits, each proved by substitution into this one

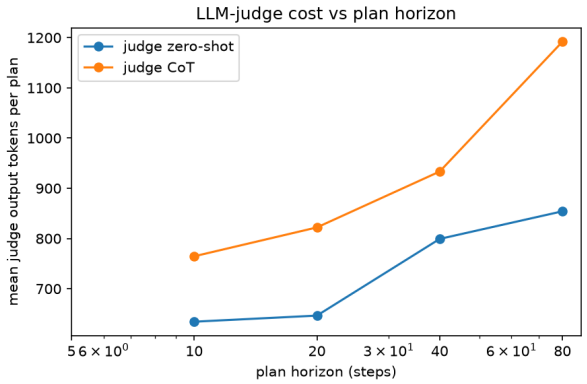

Figure 2: Mean judge output tokens vs. plan horizon. Judges spend more compute on longer plans yet still miss more flaws (Table 2).

formula (full proofs in the repository). Write c for the judge's per-record call cost, ρ for its recall, and ϕ for the checker's false-reject rate.

Prop. 1 (judge ties on accuracy). *If both have recall* 1 *and fr* 0*, checker call cost* 0*, judge* $c > 0$*, then* $\text{cost}_{\text{checker}} \leq \text{cost}_{\text{judge}}$ *for all* $S, r, \lambda \geq 0$*, equality iff* $\lambda = 0$. The S and r terms vanish, leaving $0 \leq c\lambda$: the whole gap is the judge's call cost. *Data:* the strong frontier judge at $H{=}20, 40, 80$—the checker is optimal in all $3600/3600$ grid cells (sweeping $S \in [1, 200]$, $\lambda \in [0.01, 100]$), tying only at $\lambda{=}0$.

Prop. 2 (checker has residual false-rejects). *If the checker has recall* 1 *but* $\phi > 0$ *while the judge is accuracy-perfect, the judge is optimal iff* $\lambda < \phi r/c$—a threshold independent of stakes S. *Data:* at $H{=}5$, extraction noise gives $\phi{=}0.068$; with $r{=}1$ the boundary $\lambda^{\star}{=}\phi/c{=}27$ predicts the judge optimal in the lower $\approx$85% of λ, i.e. $3060/3600$ cells—matching the measured grid, which is where the earlier grid-search result now has an algebraic explanation.

Prop. 3 (judge recall below 1). *If the checker has recall* 1*, fr* 0*, and the judge has recall* $\rho < 1$*, the checker dominates by the accuracy term alone: at* $\lambda{=}0$ *the gap is already* $(1{-}\rho)S + fr_J r > 0$ *whenever* $S > 0$—no call-cost term needed. *Data:* the weaker judge at $H \geq 10$, formalizing the original horizon-degradation observation. The three regimes are mutually exclusive on (ϕ, ρ) and each corresponds to a measured cell, so the family maps where each strategy wins without ever claiming universal dominance.

5.4 Negative Result: the Learned Trust Layer Is Inert Under Two Independent Calibration Methods

We tested a learned trust layer over the checker's parse—predicting whether the NL→schema translation was faithful—two independent ways, and it was inert both times at our data scale. We report both, with numbers.

Method 1 (Platt-scaled threshold). A calibrated logistic model (accuracy 0.958, ECE 0.067, Figure 3) does rank the signal correctly—the three false-rejected mistranslations get its *lowest* scores (0.74–0.82 vs. >0.9 elsewhere). But

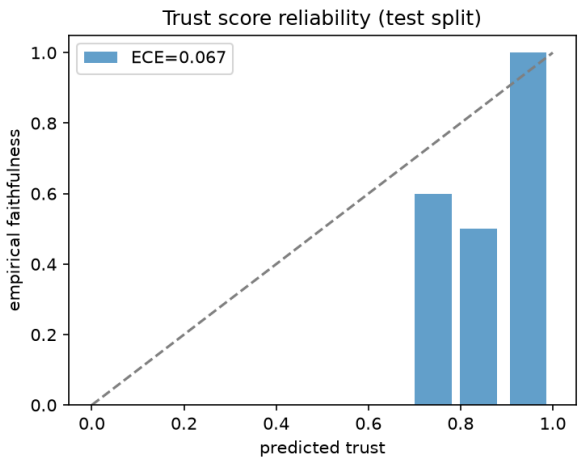

Figure 3: Trust-model reliability diagram (test split). Mass concentrates in the top bin at accuracy 1.0; the sparse low-confidence bins are exactly the mistranslated records of Section 5.4.

Arm	Success	Flawed exec.	Replans	Dead ends
No verifier	0.611	0.389	0	0
Hybrid-gated	**0.931**	**0.000**	31	**5**
CoT-judge-gated	0.875	**0.000**	28	9

Table 3: Execute-or-reject with one feedback-driven replan (72 test records). Success = an executed plan that is actually valid.

the validation-picked operating threshold is $\tau{=}0$: the layer never changes a decision, because the test split contains zero symbolically-clean-but- flawed records for it to catch (41 symbolic passes, all genuinely valid).

Method 2 (split conformal). Replacing the ad hoc threshold with a distribution-free conformal gate does not rescue it, and the reason is diagnosable. There are only 13 unfaithful-translation records total (5 calibration / 3 test after the problem-level split). At the standard miscoverage $\alpha{=}0.1$ the conformal threshold is *vacuous* ($\tau{=}1.0$, flag everything): a 90% distribution-free guarantee is simply unreachable from 5 calibration positives except by rejecting all records. At the tightest *non*-vacuous level, $\alpha{\approx}0.167$ (guaranteed coverage $5/6$), the gate flags zero symbolic-pass records and again changes nothing; realized test coverage is $3/3$ but with a Clopper–Pearson 95% interval of $[0.29, 1.0]$ on three points. Net: zero beneficial decision flips under either method. The binding limitation is the calibration-set size, not the calibration method—a concrete methodological finding about how many labeled positives split conformal needs before it can say anything (here, far more than 5).

5.5 Downstream: Execute or Reject

Table 3 is where the architecture earns its keep at this horizon. Both gates eliminate flawed execution entirely here, but the hybrid converts more rejections into successful second at-

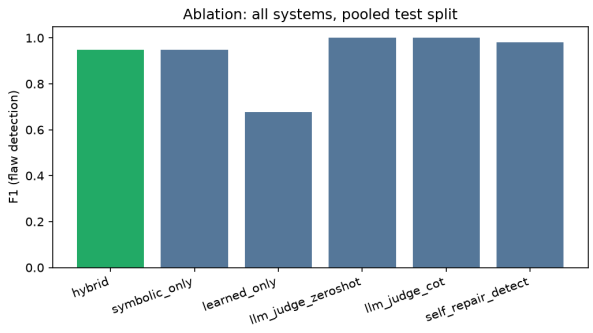

Figure 4: F1 by system and domain. The learned-only bar shows the cost of removing symbolic grounding; judge bars are at ceiling at this plan length.

tempts: its replan feedback is the symbolic explanation itself ("step 3: unknown object . . . ", "goal conjunct `on(b3,b2)` unmet"), which is more actionable than the judge's free-text critique—5 vs. 9 dead-ends, 93.1% vs. 87.5% final task success. At H=5: detection parity, feedback superiority. Section 5.2 reruns this exact experiment at longer horizons and finds the judge's "eliminates flawed execution entirely" property itself breaks down.

5.6 A Deployment Failure Mode: the Silently Absent Judge

Our first zero-shot judge used a 64-token budget, reasoning that a one-line verdict needs no more. The judge model ignored the "respond with exactly one line" instruction, simulated the plan step by step, and was truncated before ever emitting its verdict—on **360/360 records**. Because unparseable output fails open (as a naive deployment would), the judge scored P=R=0.0 while appearing to run normally: the system silently became *no verifier at all*. At 512 tokens, 41% of outputs still truncated; only at 2,048 (the CoT budget) did parse failures reach zero—whereupon the judge jumped from 0.0 to a perfect 1.0. We fixed the baseline and report the fixed numbers, but the episode is itself a finding: fail-open verdict parsing plus format non-compliance is a silent total-failure mode, and the zs-vs-CoT contrast in any judge comparison is confounded by budget unless equalized.

5.7 Self-Repair and Cost

The Reflexion-style repairer is a surprisingly strong *fixer*—23/28 flawed plans repaired, 0/44 valid plans broken—but an unreliable *detector* (change-detection conflates cosmetic rewrites with flaw detection). Cost: the symbolic path decides in $\sim$0.2 μs locally with zero API calls; the extract-and-check path adds $k\times$steps extraction calls ($\sim$12–24, batchable); each judge call generates into an 8,192-token budget (measured means 500–1,500 tokens, rising with horizon and higher for the stronger judge). A verifier that is free at decision time changes how often one can afford to check—the term the decision model prices as λ.

Domain	Step accuracy (prose)	Hyphen-mangle share
Blocksworld	0.947	0/26 (0%)
Logistics	0.939	7/24 (29%)
Tools	0.878	**50/60 (83%)**
Pooled	0.920	—

Table 4: Step-level extraction accuracy vs. the rule-based reference, and the share of same-action argument errors attributable to hyphenated-argument mangling. Confirms the mangling is the dominant extraction-error mode specifically in Tools.

5.8 Extraction Accuracy

Because the whole pipeline rests on the NL$\rightarrow$schema extraction, we report its accuracy directly, as a metric distinct from the trust model's verdict-faithfulness: the fraction of individual extracted *steps* whose (action, arguments) exactly match the rule-based reference parse (Table 4). In the prose regime, step accuracy is 0.92 pooled and is lowest on Tools (0.878). The known failure signature is confirmed quantitatively rather than anecdotally: of the same-action argument errors, **83% in Tools are hyphenated-argument mangling** (`flights-scope`$\rightarrow$`flights`), versus 0% in Blocksworld (whose errors are name normalization, `b3`$\rightarrow$"block 3") and 29% in Logistics. The constrained-format regime is lossless at the verdict level (240/240, Section 4.5); per-step extraction there was not separately logged, as the rule parser is the reference. (The real domains used deterministic injection, so step-level LLM-extraction accuracy is not reported for them.)

5.9 Generality: Synthetic and Real Domains

The Tools domain models a travel-booking agent: eight tools, auth scopes as predicates, prerequisite calls threaded as boolean return-value facts (`search_flights` adds `flight-options(trip)`, which `book_flight` requires), an API quota ($-$1/call) and a spend budget (flight 300, hotel 200) as resource dimensions. **No changes to any pipeline code were required**—the same labeler, verifier, extractor, and trust model ran as-is, and the domain's verdicts cross-check 120/120 against the oracle. The one representational wrinkle: the DSL lacks constant action arguments, so scopes are pinned by identity predicates. Notably, the pipeline's only false rejects arise here, from hyphenated tool-argument extraction—evidence that tool-style vocabularies stress the NL$\rightarrow$schema seam more than classical planning vocabularies do.

A real external benchmark (ALFWorld). To test whether the method survives contact with data we did not author, we adapted **ALFWorld** (Shridhar et al. 2021), whose task backend is PDDL. We did *not* get "zero code changes": a real benchmark needed a real adapter, with disclosed semantics-preserving compilations (a maintained `accessible` predicate for the pickup/put accessibility disjunction; existential goals grounded to the expert plan's target instances; dropped action-costs). On 80 randomly sampled real ALFRED trials, **65% (52/80) translate onto the DSL and**

pass the independent-oracle cross-check, with the remaining 35% failing for specific, reported reasons—the movable-receptacle family (nested-receptacle predicates the DSL lacks, 0/10), the knife-in-hand slice family, and look-at-in-light—not silently dropped. On the 52 translated tasks plus 154 deterministically flaw-injected variants (206 records), the labeler and checker **agree 206/206**, and the checker catches every injected flaw. Because ALFWorld's object identifiers are machine-encoded and its states have 200+ atoms, faithful LLM plan *generation* over readable names is unreliable, so flaws here are injected deterministically rather than by prompting—a disclosed methodology difference. The load-bearing result is that the sound checker and the independent oracle agree on real benchmark data, at a translation rate we report honestly rather than curate.

Real API traces. We also encoded five real public APIs (Stripe, SendGrid, GitHub, Slack, Google Calendar), with their actual OAuth scopes and prerequisite chains, into the existing Tools DSL as a labeled real-trace subset (32/32 oracle cross-check). The one representational cost, reported rather than hidden: real APIs return typed values (a payment-intent id) that later calls consume, and the DSL—lacking typed return values—models them as boolean prerequisite facts, so value-level errors are out of scope.

6 Limitations

All numbers are dev-scale: 360 short-horizon records (72-record test split, 13 negative trust labels) plus 12 records per domain per long horizon at $H \in \{40, 80\}$ (wide error bars there), one model family (`claude-haiku-4-5`) for every LLM role, single seed. The long-horizon gold plans are valid but not optimal (Section 4.2), so "horizon H" is a nominal band, not an optimality claim, and the downstream soundness-gap result (Section 5.2) is demonstrated only at $H \leq 20$ with the Haiku judge, where its recall trend suggests the gap would grow—a projection that does *not* transfer to a stronger judge. The Sonnet-5 transfer check is itself narrow: zero-shot only, three horizons, 12-record-per-domain subsets at $H = 40, 80$, and a single strong model; the accuracy tie could break for a different model, for CoT, or at larger n, and we claim only what those points show. The trust layer's value proposition is confined to the prose regime by construction (Section 4.5); the paraphrase step can itself add label noise (all systems see identical text, so comparisons stay like-for-like). The oracle and verifier share design semantics though not code, so a shared semantic error would evade the cross-check; hand-verified fixtures and (future) VAL (Howey, Long, and Fox 2004) validation are the defenses. Precondition similarity is lexical; the DSL omits conditional effects and typed return values; the downstream replan arm is gated by the sound rule-based check, so its "accepted implies valid" property is by construction and the informative quantities are replan success and ungated flawed-execution cost. A remaining open question worth naming plainly: the near-100% extraction-fidelity rate at $H \geq 10$ (Section 5.2) measures agreement between two extraction paths on mechanically repetitive long-horizon plans, not ground-truth extraction accuracy, and should not be read as translation becoming more reliable with plan length.

7 Conclusion

We built a sound pre-execution verifier for free-text LLM plans, and evaluating it corrected our own central hypothesis. We expected its edge over an LLM judge to be accuracy that widens with plan length; that held for a weaker judge model (recall $0.94 \rightarrow 0.84$ over $10 \rightarrow 80$ steps) but was overturned by a stronger frontier judge, which under the identical prompt and budget showed no measurable horizon-dependent degradation (recall 1.0 at 20, 40, and 80 steps). Horizon-driven judge degradation is a property of the judge model, not of judging.

What survives is sharper. When a judge only *matches* the checker's accuracy, the checker is still the expected-cost-optimal verification strategy, because it is sound at zero marginal decision-time cost—a dominance we prove as a proposition and confirm across the whole (stakes, cost-weight) grid at every horizon ≥ 10. The boundary is real, not universal: at 3–8 steps, where extraction noise gives the checker a small false-reject rate, a cheap accurate judge can instead be optimal, and we map exactly where. And the argument that needs no model comparison at all: the checker is structurally immune to the silent guardrail-collapse modes we document—no token budget to truncate, a total deterministic verdict, fail-closed by construction—whereas a fail-open judge with a too-small budget accepted 100% of flawed plans while appearing to run.

We also report what did *not* work: a learned trust layer over the checker's parse was inert under two independent calibration methods (Platt-scaled and split-conformal), the binding limit being a 13-example unfaithful-translation set—too few for a distribution-free guarantee to say anything. The next steps are a bidirectional fusion rule (letting low trust trigger re-extraction rather than only extra rejection), a larger calibration set, and pushing the whole comparison onto real, non-synthetic long-horizon planning tasks.

Reproducibility

All code, domain generators, prompts, and evaluation scripts are in the project repository; `make reproduce` regenerates every number and figure in this paper from fixed seeds (`make reproduce FULL=1` for the full-scale configuration). Datasets are synthesized locally; the only external dependency is an Anthropic API key.

References

Fox, M.; and Long, D. 2003. PDDL2.1: An Extension to PDDL for Expressing Temporal Planning Domains. *Journal of Artificial Intelligence Research*, 20: 61–124.

Guo, C.; Pleiss, G.; Sun, Y.; and Weinberger, K. Q. 2017. On Calibration of Modern Neural Networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*.

Howey, R.; Long, D.; and Fox, M. 2004. VAL: Automatic Plan Validation, Continuous Effects and Mixed Initiative Planning Using PDDL. In *16th IEEE International Conference on Tools with Artificial Intelligence (ICTAI)*, 294–301.

Huang, J.; Chen, X.; Mishra, S.; Zheng, H. S.; Yu, A. W.; Song, X.; and Zhou, D. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. In *International Conference on Learning Representations (ICLR)*.

Kambhampati, S.; Valmeekam, K.; Guan, L.; Verma, M.; Stechly, K.; Bhambri, S.; Saldyt, L.; and Murthy, A. 2024. LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

Liu, B.; Jiang, Y.; Zhang, X.; Liu, Q.; Zhang, S.; Biswas, J.; and Stone, P. 2023. LLM+P: Empowering Large Language Models with Optimal Planning Proficiency. *arXiv preprint arXiv:2304.11477*.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Naeini, M. P.; Cooper, G. F.; and Hauskrecht, M. 2015. Obtaining Well Calibrated Probabilities Using Bayesian Binning. In *Proceedings of the 29th AAAI Conference on Artificial Intelligence*.

Platt, J. C. 1999. Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods. In *Advances in Large Margin Classifiers*. MIT Press.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Shridhar, M.; Yuan, X.; Côté, M.-A.; Bisk, Y.; Trischler, A.; and Hausknecht, M. 2021. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. In *International Conference on Learning Representations (ICLR)*.

Stechly, K.; Valmeekam, K.; and Kambhampati, S. 2025. On the Self-Verification Limitations of Large Language Models on Reasoning and Planning Tasks. In *International Conference on Learning Representations (ICLR)*.

Valmeekam, K.; Marquez, M.; Olmo, A.; Sreedharan, S.; and Kambhampati, S. 2023. PlanBench: An Extensible Benchmark for Evaluating Large Language Models on Planning and Reasoning about Change. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*.

Wang, X.; Wei, J.; Schuurmans, D.; Le, Q.; Chi, E.; Narang, S.; Chowdhery, A.; and Zhou, D. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations (ICLR)*.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*.